

Snort RAG Rule Generator

Complete Fix Plan to Move from 78/100 to 99/100

Professional academic and implementation handoff report prepared for Codex continuation inside the project folder structure.

Strict reference used: most_important_file.pdf from Pr. Ikram BENABDELOUAHAB, FST Tanger, NLP Projects 2026. Project inspected: snort_rag_rule_generator.pdf.

Purpose. This PDF lists every material fix needed to transform the current project from a strong but incomplete academic submission into a near-perfect 99/100 submission under the instructor constraints. It is written as a direct implementation backlog for Codex.

Important honesty statement. A 99/100 target is realistic after the fixes below. A guaranteed 100/100 is not promised because grading remains subjective and depends on final execution evidence, oral defense, and the instructor's interpretation.

Source	Pages	Text chars extracted	Rendered pages verified	SHA-256
Project Context PDF	63	219,221	63	37b183d059601d296e91f0ce...
Instructor Requirements PDF	11	19,438	11	4086d584516505c717ac54a3...

Prepared after complete text extraction and render verification of all pages from both PDFs. The source PDFs were not analyzed from partial snippets only.

1. Executive Summary

The project is already above average because it contains a personal dataset, multiple retrieval strategies, a Gradio dashboard, a local Snort-like rule parser, generated artifacts, false-positive heuristics, and honest limitations. However, it cannot be considered 99/100 strict-compliant until it satisfies the instructor's central missing requirements: real LLM generation inside RAG, LLM benchmarking, clustering/family attack analysis, real/log-PCAP validation evidence, and final documentation cleanup.

The professor's Subject 2 asks for a system using NLP and RAG to automatically generate Snort rules from natural-language descriptions of suspicious network behavior. It specifically requires retrieval with Sentence-BERT/BM25, embedding benchmarking, an LLM generation module using the query plus retrieved documents, LLM benchmarking, classification and clustering, rule explanations, false-positive optimization, integration with real network logs, a dashboard with PDF upload, a personally constructed dataset, code, and a technical report. [P1]

The project currently says that it is not a direct external-LLM workflow and that generation is done by retrieved-rule selection or deterministic templates. This protects against black-box copying, but it leaves the formal LLM requirement unsatisfied. In modern RAG, the user query and retrieved information are supplied to an LLM before the final answer is generated, so a purely template-based generator should be repositioned as a baseline or fallback, not the main final module. [W4]

Area	Current status	Needed to reach 99/100	Priority
Dataset	Strong: personal, 200 rows, manual + synthetic variations, benign class.	Add dataset card, split manifest, no-leakage proof, generation provenance, and clearer synthetic generation procedure.	P1
Retrieval	Strong: BM25, TF-IDF, hybrid reranking, Sentence-BERT/FAISS artifacts.	Make best retriever the default, remove dashboard disabling of SBERT/FAISS, add k-sensitivity and error cases.	P1
Generation	Main weakness: deterministic local generator, no actual LLM call.	Add controlled LLM module inside RAG with prompt design, schema validation, repair loop, and template fallback only.	P0
LLM benchmark	Missing.	Benchmark at least 3 LLMs or 2 LLMs + deterministic baseline if hardware is limited; include metrics and analysis.	P0
Clustering	Mostly absent: t-SNE only, no cluster metrics.	Add KMeans/Agglomerative clustering, silhouette, ARI/NMI, cluster-family interpretation.	P0
Real logs / PCAP	Only a plan; no results.	Generate/capture lab PCAPs, run Snort, export alert logs, benign false-positive table.	P0
Documentation	Good but contains README merge conflict and overclaims must be avoided.	Fix merge markers, add strict compliance matrix, reproducibility commands, limitation wording.	P0

Score recovery estimate. Current strict score: 78/100. The fixes below recover approximately 21 points: LLM-in-RAG generation (+7), LLM benchmark (+4), real logs/PCAP/Snort validation (+4), clustering/family analysis (+2), expanded evaluation (+2), documentation/code polish (+2). Target after implementation: 97-99/100.

2. Source Reading Methodology and Scope

Both PDFs were read completely. The instruction PDF has 11 pages and the project context PDF has 63 pages. All pages were rendered to images and text was extracted from every page. The analysis therefore uses the complete instruction set and the complete project context, including README, requirements, docs, scripts, source modules, tests, result CSVs, and embedded result image references.

Only the professor's Subject 2 is directly applicable because the project is "Generation de regles SNORT". Other subjects were also read but are used only to understand common course expectations: personal dataset, RAG pipeline, LLM benchmarking, dashboard, methodology, and code/report quality. [P1]

2.1 Strict interpretation rules used in this audit

- 1 General instructions override project-specific convenience. The project must not use an LLM as a black-box answer generator, but it must still use an LLM inside a controlled RAG pipeline.

- 2 A personal dataset is mandatory. Public ready-made datasets must not become the main dataset. External Snort rules may be used only as separated references or optional knowledge support, not as the official corpus.
- 3 A dashboard with PDF upload is required, but uploaded PDF chunks must be clearly separated from the official personal dataset.
- 4 Real Snort runtime validation and PCAP/log tests must not be claimed unless they were actually executed and artifacts exist.
- 5 The final report must show architecture, model choices, examples, tests, limitations, improvements, and reproducibility commands.

2.2 Evidence of complete source inspection

Source	Pages	Text chars extracted	Rendered pages verified	SHA-256
Project Context PDF	63	219,221	63	37b183d059601d296e91f0ce...
Instructor Requirements PDF	11	19,438	11	4086d584516505c717ac54a3...

The project PDF includes 47 identified file/artifact sections. Key inspected areas include README, pyproject, requirements, dataset methodology, dataset validation, generation status, PCAP plan, retrieval benchmark section, rule validation notes, architecture notebook, result CSVs, benchmark scripts, validation scripts, app_gradio.py, architectures.py, evaluation.py, false_positive.py, generator.py, knowledge_base.py, retrieval.py, rule_parser.py, templates.py, run_devoir3.py, tests, and PCAP README. [P2]

3. Strict Requirement-by-Requirement Evaluation

Professor requirement	Current project evidence	Status	Required fix
No direct LLM black-box use; LLM only inside RAG with prompt control and retrieval integration.	Project avoids LLM calls entirely and uses deterministic templates/retrieved rules.	Partial	Add real LLM call after retrieval; log prompt, context, model, and output. Keep deterministic generator as fallback/baseline only.
Personal database only; no Kaggle/public ready-made dataset as main source.	Dataset is stated as personal, 200 rows, 80 manual + 120 synthetic/manual variations.	Good	Add dataset card, generation provenance, no-public-dataset statement, split manifest, and scripts that regenerate only permitted synthetic variations.
Manual data plus automatic synthetic enrichment.	Synthetic_manual_variation exists, but automatic generation procedure must be more explicit.	Mostly good	Add scripts/generate_synthetic_variations.py and docs describing controlled generation rules, seed examples, and review checklist.
Knowledge base containing Snort rules, attack descriptions, logs/signatures.	Dataset includes rules, descriptions, log_example, false_positive_context.	Good	Add rule/signature coverage table and ensure uploaded trusted rules remain separate.
Retrieval using embeddings/Sentence-BERT and BM25; benchmark embeddings.	BM25, TF-IDF, Sentence-BERT, FAISS benchmarks present.	Good	Make Sentence-BERT/FAISS installable and not disabled in dashboard; add k=1/3/5/10 and model comparison table.
Generation module uses GPT/Ne motron/Mistral/LLaMA/etc. to generate valid Snort rule from query + retrieved documents.	Missing; generator explicitly does not call LLM.	Fail	Implement src/snort_rag/llm_clients.py and RAG LLM generation/repair loop.
LLM benchmarking via evaluation metrics.	Missing.	Fail	Add scripts/benchmark_llms.py and results/llm_benchmark.csv with >=3 models where feasible.
Complete RAG: input, query encoding, Top-k retrieval, enriched prompt, generation.	Pipeline exists but final generation is template selection, not LLM generation.	Partial	Use prompt + Top-k docs as actual LLM input; save prompt and source_doc_ids.
Automatic attack type detection and attack-family clustering.	Classification exists; clustering not implemented beyond t-SNE visualization.	Partial	Add clustering module, metrics, plots, report interpretation.
Explanation of generated Snort rules.	Present, but short and sometimes generic.	Good	Expand explanations to mention header, flow, payload options, rate limits, false-positive reasoning, and retrieved evidence.
False-positive optimization.	Heuristic module exists and artifacts exist.	Good	Add empirical false-positive results from benign PCAP/log tests.
Integration with real network logs.	Only simulated logs and PCAP plan; no real execution artifacts.	Fail	Generate/capture lab PCAPs, produce Snort alert logs, export summaries.
Dashboard with PDF upload as new knowledge base.	Gradio app supports PDF upload into in-memory KB.	Good	Add KB selector, persistence option, source provenance, and LLM/model controls.
Code source complete.	Package is present with tests and scripts.	Good	Add missing modules/scripts/tests for LLM, clustering, PCAP, packaging.
Technical report with architecture, model choices, examples, limits, improvements.	Docs exist but final report must be updated after fixes.	Partial	Add strict compliance matrix, benchmark results, PCAP evidence, and limitation wording.

3.1 Non-negotiable P0 fixes

- 1 P0-01: Replace the current "no LLM" main generation with a controlled RAG+LLM generation path. The template system remains as a validated fallback and as a baseline, not as the primary final answer.
- 2 P0-02: Add LLM benchmarking with reproducible commands, metrics, and result CSVs.
- 3 P0-03: Add clustering/family attack analysis and metrics.
- 4 P0-04: Run or at least implement executable Snort runtime and PCAP/log validation scripts; to claim 99/100, actual result artifacts must be present.
- 5 P0-05: Fix README merge conflict markers before any submission.

4. P0-01 - Implement a Real Controlled LLM Generation Module

This is the most important fix. The professor explicitly asks to use a language model to generate a valid Snort rule from the user request and retrieved documents. The current project currently avoids external LLMs and generates rules using templates or retrieved context. This should be reframed as the baseline/fallback. The new main path must be RAG+LLM, with strict validation and traceability.

A correct RAG design retrieves relevant external knowledge and supplies the query plus retrieved context to an LLM before generation. This is exactly the gap to close. [W4]

4.1 Required new files

File path	Purpose	Mandatory behavior
src/snort_rag/llm_clients.py	Provider abstraction for local/API LLMs.	Defines BaseLLMClient, LocalTransformersClient, optional Ollama/OpenAI-compatible client only if permitted, and MockDeterministicClient for tests only.
src/snort_rag/prompting.py	Prompt construction and prompt variants.	Builds enriched prompt from query + Top-k documents; includes output schema, safety constraints, rule syntax reminders, no unsupported claims.
src/snort_rag/llm_generator.py	Main RAG+LLM generator.	Calls LLM, parses JSON or rule block, validates with parser, repairs once or twice, computes explanation/FP risk.
scripts/benchmark_llms.py	Run LLM comparison.	Tests multiple models on fixed evaluation set and writes results/llm_benchmark.csv plus examples.
tests/test_llm_generator.py	Unit tests for LLM generation contract.	Uses fake/mock client to ensure prompt includes retrieval context and parser handles invalid outputs.

4.2 Required modifications to existing files

Existing file	Change to make
src/snort_rag/generator.py	Split current deterministic functions into baseline/fallback utilities; do not present them as the final LLM generation module. Add a new generate_with_llm_context or delegate to llm_generator.py.
src/snort_rag/architectures.py	Add rag_llm_generate(query, k=5, model_name=...) and include it in run_all or a dedicated run_llm_bench.
src/snort_rag/app_gradio.py	Add model selector, prompt preview, retrieved document table, validation panel, repair attempts, and fallback warning if LLM unavailable.
src/snort_rag/evaluation.py	Add LLM model dimension: architecture x model. Record model_name, prompt_variant, validity, attack accuracy, FP score, latency, hallucination risk.
requirements.txt / pyproject.toml	Add optional extras for transformers, accelerate, torch, sentence-transformers, faiss-cpu, or a documented local runtime.
README.md	Replace "no LLM" wording with "LLM is used only inside controlled RAG; deterministic generator is retained as fallback/baseline".

4.3 Minimal interface specification

```
# src/snort_rag/llm_clients.py
from dataclasses import dataclass
from typing import Protocol

@dataclass
class LLMResponse:
    text: str
    model_name: str
    latency_ms: float
    prompt_tokens: int | None = None
    completion_tokens: int | None = None

class BaseLLMClient(Protocol):
    model_name: str
    def generate(self, prompt: str, *, temperature: float = 0.0, max_new_tokens: int = 512) -> LLMResponse:
        ...

# Tests must use a fake client. Production can use LocalTransformersClient or another provider.
```

4.4 Required prompt contract

The prompt must never ask the LLM to invent external knowledge. It must bind the model to the retrieved Snort examples, the local syntax constraints, and the requested output schema. The prompt must be logged in the returned result for auditability.

SYSTEM:

You are a defensive IDS rule assistant. Generate exactly one educational Snort rule. Use only the user request and the retrieved documents. Do not invent CVEs, sources, or runtime validation results. If the traffic is benign, output NO_RULE_RECOMMENDED.

RETRIEVED CONTEXT:

For each Top-k document include: id, attack_type, attack_family, log_example, snort_rule_reference, false_positive_context, expected_explanation, retrieval_score.

OUTPUT JSON ONLY:

```
{
  "attack_type": "one allowed label",
  "generated_rule": "Snort rule or NO_RULE_RECOMMENDED",
  "explanation": "short technical explanation",
  "false_positive_notes": "why it is or is not broad",
  "used_source_doc_ids": ["SNORT-P1-...."],
  "confidence": 0.0
}
```

4.5 Validation and repair loop

- 1 Call the LLM once with the enriched prompt.
- 2 Parse JSON. If JSON fails, extract a single rule between markers or ask a repair prompt.
- 3 Validate the rule using rule_parser.validate_rule. Also ensure attack_type is allowed and source_doc_ids are in retrieved docs.
- 4 If invalid, send one repair prompt containing validation errors and the original retrieved context.
- 5 If still invalid, fall back to the deterministic template and mark generation_mode="template_fallback_after_llm_failure". Do not hide fallback.
- 6 Return prompt, model_name, generation_mode, repair_attempts, validation_errors, rule, explanation, source_doc_ids, FP analysis, latency.

4.6 Done criteria

This fix is complete only when the following command produces a CSV containing multiple LLM rows, valid generated rules, model names, latencies, and prompt variants:

```
PYTHONPATH=src python scripts/benchmark_llms.py --dataset
data/processed/final_snort_dataset.csv --models local-qwen,local-mistral,local-llama
--queries data/evaluation/snort_eval_queries.csv --out results/llm_benchmark.csv
```

If hardware prevents running three local LLMs, the report must state the limitation clearly and compare at least one real LLM, the deterministic baseline, and retrieved-rule baseline. However, for a 99/100 target, at least three LLM entries is strongly recommended because the instructor repeatedly asks for LLM benchmarking in the project file. [P1]

5. P0-02 - Add LLM Benchmarking and Prompt Benchmarking

The current project benchmarks retrieval methods and RAG architectures, but not LLMs. The professor explicitly requires benchmarking of LLMs via evaluation metrics. The benchmark must be reproducible, stored under results, and explained in the report.

5.1 Required metrics

Metric	Definition	Why it matters
valid_rule_rate	Share of outputs accepted by local parser and, when available, Snort runtime.	Directly measures rule validity.
attack_type_accuracy	Predicted label equals expected label.	Measures classification quality.
retrieval_grounding_rate	Every used_source_doc_id is in Top-k and the rule content is consistent with retrieved class.	Prevents hallucination.
option_coverage	Presence of required options such as msg, sid, rev, classtype, flow/content/pcrc/detection_filter as applicable.	Measures Snort completeness.
false_positive_score	Heuristic score plus empirical benign PCAP/log triggers after PCAP testing.	Measures alert quality.
runtime_snort_pass_rate	Share of generated rules accepted by real Snort validation command.	Required for stronger claims.
pcap_detection_rate	Expected attack PCAPs that trigger correct SID/category.	Measures behavior, not just syntax.
benign_alert_rate	Benign PCAPs that trigger generated rules.	Empirical false-positive check.
latency_ms	End-to-end generation time.	Performance and dashboard usability.

5.2 Required outputs

- results/llm_benchmark.csv - one row per query x model x prompt_variant.
- results/llm_benchmark_summary.csv - aggregate metrics by model.
- results/llm_generated_examples.csv - selected good and bad examples.
- results/prompt_benchmark.csv - compare prompt_v1, prompt_v2, prompt_strict_json.
- docs/report_section_llm_benchmark.md - methodology, table, analysis, limitations.

5.3 Evaluation-set fix

The current evaluation set contains only 10 queries, one per class. That is too small for a near-perfect submission and may inflate metrics. Create data/evaluation/snort_eval_queries.csv with at least 80-120 queries, balanced across the 10 labels, including English/French variants, short informal requests, ambiguous requests, benign traffic, and adversarial but defensive wording. Keep this evaluation file separate from the retrieval dataset to reduce leakage.

```
# Required columns for data/evaluation/snort_eval_queries.csv
id,query,expected_attack_type,expected_family,language,difficulty,notes
EV-001,"Detect SQL injection with UNION SELECT in HTTP URI",sql_injection,web,EN,medium,"web
payload"
EV-002,"Traffic is a normal health check, no attack",benign_traffic,benign,EN,easy,"negative
example"
```

6. P0-03 - Add Attack-Family Clustering

The professor requires automatic attack-type detection and attack-family clustering. The project has classification and t-SNE visualization, but t-SNE is visualization, not clustering evaluation. Add explicit clustering over document embeddings and report whether clusters align with attack families.

6.1 Required files

File path	Purpose
src/snort_rag/clustering.py	Functions to compute embeddings, run KMeans/Agglomerative clustering, map clusters to labels, and compute metrics.
scripts/run_clustering_analysis.py	CLI that generates results/clustering_metrics.csv and plots.
results/clustering_metrics.csv	silhouette, ARI, NMI, homogeneity, completeness, v_measure.
results/clustering_confusion_matrix.csv	Cluster vs attack_type/family table.
results/clustering_tsne.png	Visualization colored by predicted cluster and labeled by true class.
docs/report_section_clustering.md	Methodology and interpretation.

6.2 Suggested implementation

```
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score, normalized_mutual_info_score

# Compare at minimum:
# 1) TF-IDF + KMeans(n_clusters=10)
# 2) Sentence-BERT + KMeans(n_clusters=10)
# 3) Sentence-BERT + AgglomerativeClustering(n_clusters=10)
# Report both attack_type and attack_family agreement.
```

6.3 Done criteria

- The report contains a table showing clustering metrics and one paragraph interpreting why some classes cluster together or overlap.
- The dashboard optionally shows a cluster/family visualization or at least a saved clustering plot.
- The code does not confuse t-SNE with clustering. t-SNE can be visualization only.

7. P0-04 - Add Real Snort Runtime, PCAP, and Log Evidence

The project correctly states that PCAP tests are future work and that local validation is not equivalent to real Snort runtime validation. For a 99/100 submission, this must move from plan to executed evidence. Snort rule basics require a rule header and body, and real Snort evaluation is the stronger authority for syntax/runtime behavior. [W1]

The official Snort documentation explains that rule options are essential for targeting the correct traffic; therefore, local string checks alone are not enough for final behavioral claims. [W1]

7.1 Required new scripts and artifacts

Path	Required content
scripts/generate_lab_pcaps.py	Use Scapy or tcpreplay-friendly generation to create small lab PCAPs for each attack class and benign controls.
scripts/run_snort_validation.py	Run Snort rule validation with --warn-all/--pedantic or equivalent. Export stdout/stderr and parsed pass/fail.
scripts/run_pcap_tests.py	Run Snort on each PCAP with data/processed/person1_rules.rules and collect alerts/SIDs.
tests/pcaps/README.md	Expand with exact generation command, privacy warning, and expected PCAP list.
results/snort_runtime_validation.csv	Rule/SID-level runtime validation status.
results/pcap_test_results.csv	PCAP, expected attack, alert count, triggered SID, pass/fail, false positives.
results/snort_alert_logs/	Sanitized alert_fast logs or parsed summaries.
docs/report_section_pcap_runtime_validation.md	Executed commands, environment, results, limitations.

7.2 Required PCAP matrix

Scenario	Attack PCAP	Benign control	Expected result
port_scan	Repeated TCP SYN to multiple ports.	Few isolated normal SYNs.	Attack alerts; benign no alert.
ssh_bruteforce	Repeated SSH connections to 22/2222.	Single admin SSH login.	Attack alerts after rate threshold.
sql_injection	HTTP request containing UNION SELECT / OR 1=1.	Legitimate page with harmless word select.	Attack alert; benign ideally no alert.
xss	HTTP URI/body with <script> or onerror.	Normal HTML content.	Attack alert; benign no alert.
command_injection	HTTP param with ;wget, &&curl, whoami.	Documentation page containing shell examples.	Attack alert; benign no alert or flagged as FP if broad.
directory_traversal	URI with ../, %2f, /etc/passwd, win.ini.	Normal nested paths.	Attack alert; benign no alert.
dns_tunneling	Long/repetitive encoded DNS queries.	Long but legitimate CDN/telemetry domains.	Attack alert; benign no alert.
icmp_sweep	Echo requests to many internal hosts.	Few monitoring pings.	Attack alert after threshold.
malware_c2	Periodic /gate.php or /beacon egress.	Normal API/health sync traffic.	Attack alert; benign no alert.
benign_traffic	Not applicable.	Mixed normal web/DNS/SSH/ICMP.	No alerts or documented FP.

7.3 Commands to implement and document

```
# Generate lab PCAPs. Do not use sensitive real traffic.
PYTHONPATH=src python scripts/generate_lab_pcaps.py --out tests/pcaps/generated
```

```
# Validate all rules with a real Snort runtime.
PYTHONPATH=src python scripts/run_snort_validation.py --snort-bin snort --config
"$SNORT_CONFIG" --rules data/processed/person1_rules.rules --out
results/snort_runtime_validation.csv
```

```
# Replay PCAPs and collect alerts.
PYTHONPATH=src python scripts/run_pcap_tests.py --snort-bin snort --config "$SNORT_CONFIG"
```

```
--rules data/processed/person1_rules.rules --pcap-dir tests/pcaps/generated --out  
results/pcap_test_results.csv
```

7.4 Important reporting rule

If Snort is unavailable, the report must say: "Runtime Snort validation was not executed in this environment; only local structural validation was executed." However, for the 99/100 target, Codex should implement the scripts and the student should execute them in a Snort-capable environment before submission.

The `detection_filter` option is particularly relevant for repeated events such as brute force, port scans, ICMP sweeps, and beaconing because it requires multiple rule hits before generating an event. The implementation should validate its use and report thresholds. [W3]

8. P1 - Dataset and Knowledge-Base Fixes

The dataset is one of the best parts of the project, but several additions can turn it from "good" to "excellent" in academic grading. The goal is not to replace the dataset; it is to strengthen provenance, traceability, split discipline, and demonstration of manual/synthetic construction.

8.1 Required dataset documentation

Artifact	Add / fix
data/processed/DATASET_CARD.md	Describe purpose, labels, columns, manual creation, synthetic enrichment, languages, limitations, intended use, and non-production warning.
data/processed/dataset_manifest.json	Rows count, label distribution, source_type distribution, SHA-256 hashes of CSV/JSONL/rules, validation date, script version.
data/evaluation/snort_eval_queries.csv	Separate evaluation queries not copied from dataset descriptions.
docs/dataset_methodology.md	Add exact synthetic-generation protocol and manual review checklist.
docs/dataset_validation_report.md	Add evidence that no public pre-labeled dataset was imported as the official dataset.

8.2 Dataset quality checks to add

- No duplicate descriptions or near-duplicates above a defined similarity threshold.
- No duplicate SID among malicious rules.
- Benign rows always use NO_RULE_RECOMMENDED.
- Every malicious row has at least one detection discriminator: content, pcre, flags, dsize, detection_filter, or equivalent.
- Each class has positive and negative evaluation examples.
- Source separation: personal dataset vs trusted external rule knowledge base vs uploaded PDFs.

8.3 Automatic synthetic enrichment without violating the rule

The professor requires synthetic enrichment, but also forbids ready-made public datasets as the main dataset. The safest approach is controlled variation of personal seed examples. Codex should add a script that uses templates/rules, random seeds, and manual review flags rather than scraping a public dataset.

```
# Suggested columns to add or verify
id, description_naturelle, attack_type, attack_family, severity, protocol, src_port, dst_port,
log_example, snort_rule_reference, false_positive_context, expected_explanation,
source_type, synthetic_seed_id, reviewed_by_human, review_notes
```

9. P1 - Retrieval and Embedding Benchmark Improvements

Retrieval is already strong, but several details should be fixed to avoid losing points. The professor asks for embeddings and BM25 with benchmarking. The project has this, but the dashboard currently forces `sentence_transformers` and `faiss` modules to `None`, which disables true dense/FAISS paths inside the app.

9.1 Required fixes

Issue	Fix
Dashboard disables <code>sentence_transformers/faiss</code> with <code>sys.modules.setdefault</code> .	Remove that code. Instead, handle <code>ImportError</code> gracefully in <code>retrieval.py</code> and display backend availability in the UI.
<code>requirements.txt</code> comments make Sentence-BERT/FAISS optional.	Use extras: <code>pip install -e .[embeddings]</code> or include a clear command. For final benchmark reproducibility, the environment must install them.
Best method not necessarily default everywhere.	Use <code>hybrid_rerank_best</code> or agentic path with hybrid reranking as default in app and report.
Only summary rows shown in PDF.	Commit full benchmark CSVs, plots, and backend proof JSON. Ensure no truncated artifacts in report.
No retrieval error analysis.	Add top-k failure cases, ambiguous query examples, and why reranking helps.

9.2 Additional retrieval metrics

- Hit@1, Hit@3, Hit@5, Hit@10.
- Precision@k and Recall@k by `attack_type`.
- MRR and average latency.
- Failure-case examples with retrieved doc IDs.
- Backend proof: `sentence_bert_runtime_available`, `faiss_runtime_available`, model name, index type.

Sentence-BERT is appropriate because it produces semantically meaningful sentence embeddings that can be compared with cosine similarity and is designed for semantic similarity search and clustering. [W5] Faiss is appropriate for efficient similarity search and clustering of dense vectors. [W6]

10. P1 - Snort Rule Quality and Parser Improvements

The rule generation is educational and structurally validated. To reach 99/100, make the rules closer to Snort 3 best practices and make the parser stricter. Official Snort documentation distinguishes the rule header from the rule body and emphasizes that options make the rule target the right traffic. [W1]

10.1 Parser improvements

Parser area	Required improvement
Rule grammar	Parse action, protocol, source net, source port, direction, destination net, destination port, and option body into structured fields.
Option syntax	Check that options are separated by semicolons and key/value options use colon where required.
Required metadata	msg, sid, rev, classtype required for malicious rules.
HTTP web attacks	Require flow:to_server,established and an HTTP sticky buffer or service:http where appropriate.
Rate-based attacks	Require detection_filter for brute force, port_scan, icmp_sweep, malware_c2 beaconing unless justified.
Benign handling	NO_RULE_RECOMMENDED is valid only when attack_type is benign_traffic.
Duplicate SID	Validate uniqueness across generated and exported rules.
Severity mapping	Ensure classtype aligns with attack_family and severity.

10.2 Snort 3 HTTP buffer improvements

For URI-based web attacks, prefer explicit HTTP sticky buffers. Snort 3 supports http_uri and http_raw_uri; http_uri searches normalized URI data, while http_raw_uri searches the unnormalized URI and is useful for percent-encoded traversal. [W2]

```
# Example direction for templates/LLM validation, not a fixed final rule:
# SQLi in normalized query:
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS (
  msg:"LOCAL SQL injection attempt";
  flow:to_server,established;
  http_uri:query;
  content:"union select"; nocase;
  classtype:web-application-attack;
  sid:91xxxxx; rev:1;
)

# Directory traversal encoded/raw:
# Consider http_raw_uri when testing for ../ or %2e%2e/%2f variants.
```

10.3 Fix current generated-rule weaknesses

Current observation	Risk	Fix
Port scan rule uses any destination port and no content/pcrc.	Broad by design, but should be empirically controlled by detection_filter and PCAP tests.	Keep rate logic; add benign SYN controls and document why no payload content is expected for SYN scans. Do not blindly require content for port scans.
Some web rules use \$HOME_NET 80 instead of \$HTTP_SERVERS \$HTTP_PORTS.	Less general and less consistent.	Normalize template/LLM post-processing to \$HTTP_SERVERS \$HTTP_PORTS when appropriate.
SQLi example lacks http_uri in retrieved dataset rule.	May be less precise for web URI matching.	Improve dataset rules or post-process LLM output to include HTTP buffer where appropriate.
DNS tunneling explanation shows classtype mismatch in example: rule trojan-activity, explanation policy-violation.	Documentation inconsistency.	Fix CLASSTYPE mapping and explanation generation to use actual parsed classtype from rule.
Generated example duplicates source IDs in some cases.	Weak retrieval trace.	Deduplicate source_doc_ids while preserving order.

11. P1 - Dashboard Improvements

The dashboard already satisfies the basic requirement of a web interface with PDF upload. To reach a near-perfect grade, it should become a transparent RAG evaluation dashboard, not just a form that returns a rule.

11.1 Add UI components

- Model selector: deterministic baseline, local Mistral, local LLaMA/Qwen, etc.
- Retrieval selector: BM25, TF-IDF, hybrid, hybrid+rerank, Sentence-BERT, FAISS.
- Top-k retrieved documents table: id, attack_type, score, source, short text, rule reference.
- Prompt preview panel showing enriched prompt used by LLM.
- Generated JSON/raw LLM output panel for debugging.
- Validation panel: local parser result, missing options, Snort runtime result if available.
- False-positive panel: heuristic factors and empirical benign PCAP result if available.
- PDF knowledge-base manager: uploaded docs listed separately from official personal dataset.
- Export button: save generated rule and metadata to results/manual_tests/YYYYMMDD.json.

11.2 Critical bug/anti-pattern to remove

Do not force sentence_transformers or faiss to None inside app_gradio.py. That hides optional backends even when installed. Instead, expose backend status in the UI and fall back only if ImportError occurs naturally.

12. P1 - Evaluation, Testing, and Reproducibility

The current tests are useful but too small for a 99/100 target. Expand testing to cover LLM schema, prompt grounding, clustering, real Snort validation, and PCAP outcomes. Also create a single reproducibility command that rebuilds all results.

12.1 Test suite additions

Test file	Minimum assertions
tests/test_llm_generator.py	Prompt includes Top-k doc IDs; generated output is parsed; invalid LLM output enters repair; fallback is explicitly marked.
tests/test_prompting.py	No prompt is built without retrieved context for RAG path; benign prompt allows NO_RULE_RECOMMENDED.
tests/test_clustering.py	Cluster metrics file is produced and contains expected columns.
tests/test_runtime_scripts.py	Snort scripts handle missing snort binary as SKIPPED, not fake PASS.
tests/test_pcap_scripts.py	PCAP generation creates files or metadata; no sensitive pcap committed accidentally.
tests/test_dashboard_contract.py	generate() returns rule, attack_type, explanation, retrieved docs, validation and model metadata.

12.2 Reproducibility command

```
# scripts/reproduce_all.sh should run these in order:
python scripts/validate_dataset.py
python scripts/benchmark_retrieval.py
python scripts/plot_retrieval_benchmark.py
python -m snort_rag.run_devoir3
python scripts/run_clustering_analysis.py
python scripts/benchmark_llms.py --models ...
python scripts/generate_generation_examples.py
python scripts/generate_lab_pcaps.py --out tests/pcaps/generated
python scripts/run_snort_validation.py --out results/snort_runtime_validation.csv || true
python scripts/run_pcap_tests.py --out results/pcap_test_results.csv || true
pytest tests/
```

If Snort is not installed, the reproduction script may mark runtime/PCAP sections as SKIPPED, but the final submitted report must not present skipped results as successful runtime validation.

13. P0/P1 - Documentation and Report Fixes

13.1 README immediate fix

The README contains unresolved Git conflict markers. This is a visible professionalism problem and must be fixed before any other submission packaging.

```
# Replace this bad block:
<<<<<<<< HEAD
# Snort RAG Rule Generator
=====
# Snort RAG Rule Generator - NLP Mini Project
>>>>>>> 99ef1088bcb774fb2474c8cf999a3d20daa0fe65

# With exactly one clean title:
# Snort RAG Rule Generator - NLP/RAG Mini Project
```

13.2 Report sections to add or rewrite

Report section	Required content
Strict compliance matrix	Map each professor requirement to project artifact and status.
LLM generation module	Explain model(s), prompt design, retrieval grounding, output schema, validation/repair, and fallback.
LLM benchmark	Table by model and prompt with metrics, qualitative examples, and analysis.
Clustering	Explain embeddings, clustering method, metrics, plots, and limitations.
Runtime validation	Show Snort command, environment, rule count, warnings/errors, pass/fail.
PCAP/log validation	Show PCAP list, attack/benign scenarios, alert counts, SIDs, false positives.
Limits and improvements	Keep honest: local/educational rules unless runtime and PCAP evidence exists.
Ethics and defensive scope	State that rules are for defensive IDS education and not production deployment without review.

13.3 Wording that must be corrected

Current wording/problem	Replace with
"This is not a direct external-LLM generation workflow."	"The final module uses an LLM only inside a controlled RAG pipeline; deterministic templates remain a baseline and fallback."
"APIs OpenAI/Mistral/Ollama are forbidden" in notebook.	Clarify: "Direct black-box use is forbidden; LLM use is allowed only through the controlled RAG pipeline required by the assignment. Local/offline models are preferred if APIs are not permitted."
"PCAP validation future work" after implementation.	If executed: "PCAP validation was executed on lab-generated traffic; results are summarized in results/pcap_test_results.csv." If not executed: keep future-work wording.
Perfect metrics without caveat.	State that initial metrics were on a small balanced test set and expanded evaluation was added for robustness.

14. Security, Ethics, and Academic Integrity Guardrails

This is a defensive cybersecurity project. Codex should avoid turning the project into an attack-generation tool. Payload strings used for rule matching and PCAP simulation should be minimal, controlled, and educational. The system should generate detection logic, not exploitation instructions.

- Do not add exploit automation, scanning tools, credential brute-force tooling, or instructions for real-world offensive use.
- Lab PCAPs must be synthetic or anonymized. Do not commit sensitive real traffic.
- LLM output must be validated before display as a rule. Invalid outputs should not be silently accepted.
- Never claim production-readiness. State that final operational deployment requires expert review, real network testing, and environment-specific tuning.
- Preserve the academic constraint: the official dataset is personally constructed, not imported from public ready-made datasets.

15. Codex Implementation Backlog

This section is written as a direct handoff. Implement in order. Each task has files, exact action, and done criteria.

ID	Task	Files	Action	Done criteria
P0-01	Fix README conflict	README.md	Remove conflict markers, clean title, update no-LLM wording.	README has no <<<<<<, =====, >>>>>> markers.
P0-02	Add LLM client abstraction	src/snort_rag/llm_clients.py	Create provider interface, response dataclass, fake client for tests, optional local client.	Unit tests pass without real model.
P0-03	Add prompt builder	src/snort_rag/prompting.py	Build strict prompt from query + retrieved docs + schema.	Prompt includes doc IDs, rules, log context, and JSON schema.
P0-04	Add RAG+LLM generator	src/snort_rag/llm_generator.py; generator.py	Call LLM, parse output, validate, repair, fallback with explicit mode.	Generated result contains model_name, generation_mode, repair_attempts.
P0-05	Wire LLM into architectures	src/snort_rag/architectures.py	Add rag_llm_generate and optionally run_all_llm.	Evaluation can call RAG+LLM path.
P0-06	Add LLM benchmark	scripts/benchmark_llms.py; src/snort_rag/evaluation.py	Run multiple models/prompts over evaluation set.	results/llm_benchmark.csv and summary produced.
P0-07	Add clustering	src/snort_rag/clustering.py; scripts/run_clustering_analysis.py	KMeans/Agglomerative + metrics/plots.	results/clustering_metrics.csv exists with silhouette/ARI/NMI.
P0-08	Add runtime Snort validation	scripts/run_snort_validation.py	Run Snort validation and parse outputs.	results/snort_runtime_validation.csv exists; missing binary becomes SKIPPED.
P0-09	Add PCAP/log testing	scripts/generate_lab_pcaps.py; scripts/run_pcap_tests.py	Create lab PCAPs and replay through Snort.	results/pcap_test_results.csv + logs exist after Snort run.
P1-01	Improve dashboard	src/snort_rag/app_gradio.py	Model/retrieval selector, prompt, validation, FP panel, PDF KB provenance.	Dashboard exposes RAG internals and no longer disables SBERT/FAISS.
P1-02	Strengthen dataset docs	data/processed/DATASET_CARD.md; dataset_manifest.json; docs/*.md	Add data provenance, split, validation hashes, review checklist.	Dataset documentation can be read independently.
P1-03	Expand evaluation set	data/evaluation/snort_eval_queries.csv	80-120 balanced queries; separate from dataset.	Metrics no longer rely on 10 toy queries only.
P1-04	Improve parser/templates	rule_parser.py; templates.py	Snort 3 buffers, metadata, parser structure, classtype consistency.	Rules have fewer missing options and better web specificity.
P1-05	Update final report docs	docs/report_section_*.md; rapport_*.docx/pdf	Add compliance matrix, LLM benchmark, clustering, PCAP/runtime results.	Final report no longer contradicts professor requirements.
P2-01	Submission packaging	scripts/make_submission_package.py	Bundle code, dataset, results, report, README, without sensitive raw traffic.	Reproducible zip/tar package generated.

15.1 Suggested execution order

- 1 Fix README and documentation contradictions immediately.
- 2 Implement LLM abstraction, prompt builder, and RAG+LLM generator using a mock client first.
- 3 Add tests for LLM prompt and output parsing, then wire real local model support.
- 4 Implement LLM benchmark and expanded evaluation set.
- 5 Add clustering analysis.
- 6 Improve parser/templates and fix consistency issues discovered by benchmarks.
- 7 Implement PCAP generation and Snort runtime scripts.
- 8 Update dashboard to expose model/retrieval/validation details.

- 9 Regenerate all results and update final report.
- 10 Create submission package and verify no sensitive PCAP/logs are included unless they are lab-generated and intended for submission.

16. Final Acceptance Criteria for 99/100

Codex should not stop at code compilation. The project should be considered 99/100-ready only if every item below is true.

Category	Acceptance criterion
LLM in RAG	At least one real LLM path generates rules using retrieved Top-k context and a strict prompt. Template fallback is explicit, not hidden.
LLM benchmark	results/llm_benchmark.csv and summary exist; report compares models and metrics.
Prompt design	Prompt variants are documented and benchmarked; prompt includes retrieved docs and output schema.
Retrieval	BM25, Sentence-BERT, FAISS/embedding paths are benchmarked; best method justified.
Dataset	Personal dataset is documented with card, manifest, labels, manual/synthetic split, and validation result.
Clustering	Clustering metrics and plot exist; report interprets attack-family clustering.
Snort validation	Runtime validation script exists; actual pass/fail artifact exists if Snort available. No fake pass.
PCAP/logs	Lab PCAP and alert-log results exist or are honestly marked not executed; for 99 target, execute them.
Dashboard	Dashboard has PDF upload, model selector, retrieval selector, prompt/context visibility, validation and FP metadata.
Report	Final report includes strict compliance matrix, architecture, choices, examples, metrics, limitations, improvements.
Code quality	No merge conflicts, no broken imports, no hidden disabling of optional backends, tests pass.
Academic integrity	No direct copy/paste LLM answer workflow; LLM only used through designed RAG pipeline; no public dataset as main dataset.

16.1 Final command checklist

```
# Must pass before submission
PYTHONPATH=src pytest tests/
PYTHONPATH=src python scripts/validate_dataset.py
PYTHONPATH=src python scripts/benchmark_retrieval.py
PYTHONPATH=src python scripts/run_clustering_analysis.py
PYTHONPATH=src python scripts/benchmark_llms.py --models <configured_models>
PYTHONPATH=src python -m snort_rag.run_devoir3
PYTHONPATH=src python scripts/generate_generation_examples.py
PYTHONPATH=src python scripts/run_snort_validation.py --rules data/processed/person1_rules.rules
PYTHONPATH=src python scripts/run_pcap_tests.py --pcap-dir tests/pcaps/generated
python -m snort_rag.app_gradio
```

17. Detailed Findings from the Project PDF

Finding	Evidence in project context	Impact	Fix
README conflict markers	README starts with <<<<<<<< HEAD / ===== / >>>>>>>>.	Major polish issue.	Clean merge conflict.
No actual LLM generation	README and docs say no OpenAI/Claude/Mistral/Ollama and prompt is not sent to an API.	Major compliance gap.	Add controlled RAG+LLM path.
Baseline named llm_no_rag but no LLM	architectures.py function llm_no_rag uses keyword detection/template.	Misleading naming.	Rename to baseline_no_rag or make actual LLM baseline optional.
PCAP tests not executed	docs/pcap_testing_plan says no PCAP result should be claimed until run.	Missing required real/log integration.	Implement and execute PCAP/Snort scripts.
Snort runtime not installed in environment	rule_validation_notes says only structural validation executed.	Syntax/runtime claims limited.	Run Snort in Docker/local lab and store results.
Only 10 evaluation queries	evaluation.py TEST_QUERIES has one per class.	Metrics may be inflated.	Add 80-120 query evaluation file.
Clustering missing	No clustering module found; t-SNE exists only.	Professor requirement incomplete.	Add clustering.py and metrics.
Dashboard disables SBERT/FAISS	app_gradio.py sets sentence_transformers/faiss modules to None.	Undermines embedding benchmark in live app.	Remove this and show backend status.
Generation examples show some inconsistencies	DNS rule/explanation classtype mismatch; duplicate source IDs.	Documentation credibility risk.	Parse actual classtype and deduplicate source IDs.
Template fallback too central	generator.py chooses retrieved rule or generate_snort_rule template.	Fails LLM generation expectation.	Keep as fallback only.

18. Recommended Final Project Structure After Fixes

```
snort_rag_rule_generator/  
  data/  
    processed/  
      final_snort_dataset.csv  
      final_snort_dataset.jsonl  
      person1_rules.rules  
      DATASET_CARD.md  
      dataset_manifest.json  
    evaluation/  
      snort_eval_queries.csv  
  docs/  
    report_section_dataset_kb.md  
    report_section_retrieval_benchmark.md  
    report_section_generation_validation.md  
    report_section_llm_benchmark.md  
    report_section_clustering.md  
    report_section_pcap_runtime_validation.md  
    strict_compliance_matrix.md  
  results/  
    comparison_metrics.csv  
    detailed_devoir3_results.csv  
    embedding_benchmark.csv  
    retrieval_benchmark_k3_summary.csv  
    llm_benchmark.csv  
    llm_benchmark_summary.csv  
    clustering_metrics.csv  
    clustering_confusion_matrix.csv  
    snort_runtime_validation.csv  
    pcap_test_results.csv  
    generated_rule_examples.csv  
    false_positive_analysis.csv  
    snort_alert_logs/  
  scripts/  
    benchmark_retrieval.py  
    benchmark_llms.py  
    run_clustering_analysis.py  
    generate_lab_pcaps.py  
    run_snort_validation.py  
    run_pcap_tests.py  
    reproduce_all.sh  
    make_submission_package.py  
  src/snort_rag/  
    app_gradio.py  
    architectures.py  
    clustering.py  
    evaluation.py  
    false_positive.py  
    generator.py  
    llm_clients.py  
    llm_generator.py  
    prompting.py  
    knowledge_base.py  
    retrieval.py  
    rule_parser.py  
    templates.py  
  tests/  
    test_llm_generator.py  
    test_prompting.py  
    test_clustering.py  
    test_runtime_scripts.py  
    test_pcap_scripts.py  
    test_generator.py  
    test_retrieval.py  
    test_rule_parser.py
```

19. Sources and References

Internal source PDFs read completely

- [P1] most_important_file.pdf - NLP Projects 2026, Pr. Ikram BENABDELOUAHAB, FST Tanger. 11 pages read completely. Main applicable section: Subject 2, Generation de regles SNORT, plus general instructions.
- [P2] snort_rag_rule_generator.pdf - Project Context: snort_rag_rule_generator. 63 pages read completely, including project tree, code files, docs, scripts, tests, and results.

External technical references used to align fixes with accepted definitions/tools

- [W1] Snort 3 Rule Writing Guide, The Basics - rule header/body and option structure.
<https://docs.snort.org/rules/>
- [W2] Snort 3 Rule Writing Guide, http_uri and http_raw_uri sticky buffers.
<https://docs.snort.org/rules/options/payload/http/uri>
- [W3] Snort 3 Rule Writing Guide, detection_filter option.
https://docs.snort.org/rules/options/post/detection_filter
- [W4] AWS, What is Retrieval-Augmented Generation?
<https://aws.amazon.com/what-is/retrieval-augmented-generation/>
- [W5] Reimers and Gurevych, Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.
<https://arxiv.org/abs/1908.10084>
- [W6] FAISS documentation. <https://faiss.ai/index.html>

20. Final Note to Codex

Do not make the project look artificially perfect. The final grade improves most when the code produces auditable evidence: model names, prompts, retrieved sources, validation errors, rule outputs, benchmark tables, clustering metrics, and PCAP/Snort results. Implement transparency first, then polish the report. The instructor's instructions reward understanding of the complete RAG architecture, not simply producing a good-looking final rule.